ASSIGNMENT - 01

01 What is R programming

→ R is an open source programming language used for statistical analysis and graphics representation and reporting.

02 What are the uses of R programming?

→ * Statistics and data analysis
* Data science and machine learning
* Business and Finance
* Health care and medicine.

03 List the basic data types of R programming

→ 01. logical
02. Numeric
03. Integer
04. Complex
05. Character
06. Raw

04 What are non-numeric values? List them

→ Non-numeric values can be used to mark abnormal or missing values in a vectors arrays or other data structure.

non-numeric values : 01. infinity
02. NaN (Not a Number)
03. NA (Not Available)
04. NULL

05 What is vector in R? Give an example.

→ A vector is simply a list of items that are of same type. to combine the list of items to a vector use the c() and seperate elements by comma

Ex: v1 ← c(2, 6, 8, 5)

06 Define list in R. Give an example

→ Lists are one-dimensional, ~~heteg~~ heterogeneous data structure. List of matrix, list of vector, list of charact. list of functions so on.

Ex: List1 ← list (24, 99, 34)

07 What is data frame in R? give example

→ A data frame is a two dimentional data structure which are store data in tabular column

Ex: df1 ← data.frame (name = c ("Sachin", "virat"), Age = c (40, 35), Runs = c(22000, 18000)) 1~

print (df1)

O/P

name	Age	Runs
Sachin	42	22000
virat	35	18000

08 List special values in R. Mention its significance

→ 01. Infinity → is.finite() & is.infinite()

02. NaN → is.nan()

03. NA → is.na()

04. NULL → is.null()

09 Differentiate between matrix and array in R-programming

→

Matrix	Array
* A matrix is a two dimensio-nal data structure where data is arranged into rows and colums	* In R, arrays are the data structures which allows us to store same type of data in more than two dimentions
Ex: m1 ← matrix (c(1,2,3,4), nrow = 2, ncol = 2)	Ex: array1 ← array (c(1:12), dim = c(2,3,2))

10. What is coercion in R? Give an suitable example

→ Converting from one object / datatype to another is refenred as as coercion

ex: a ← c(1, 2, 0, 3) as.numeric(a) ⇒ 1 2 0 3

as.logical(a) ⇒ T T F T

11 Mention any four plotting functions in R

→ * the plot()
* ggplot()
* barplot()
* hist()
* boxplot()

12 List anithmetic operators with examples in R

→ * + Ex: 3+5 o/p 8
* - Ex: 8-3 o/p 5
* * Ex: 2*2 o/p 4
* %% Ex: 2%%3 o/p 2

13 Mention the various assignment operators in R

→ * equal operator (=)
* using left assignment / leftward operator (← or <<-)
* using right assignment / rightward operator (→ or →>)

14 Define classes in R.

→ classes are blueprint which are responsible for providing different properties and methods to perform operations

15 What is leftward and rightward assignment operator?

→ leftward assignment: By using left assignment operators you can assign right side value to left hand side variable

rightward assignment: By using right assignment operator you can assignment left side value to the right hand side variable.

16 Define rev() & seq() functions in R

→ rev(): This function reverse the vector elements

ex: v1 ← c(1,2,3) print(rev(v1)) ⇒ 3,2,1

Seq(): In R programming seq() is used to create a sequence of element in a vector. ex: v1 ← seq(c(1:10), by=2)

⇒ 1 3 5 7 9

17 Difference between class() and typeof() functions in R

→ *class() is a built-in function used to determine the datatype of the variable.

*type of() can be used to access the type of an object while type is one of the three properties of a vector

18 Define matrix with an example.

→ A matrix is a two dimentional data structure where data is arranged into rows and columns.

Ex: m1 ← matrix(c(1,2,3,4), nrow=2, ncol=2)

print(m1)

[,1] [,2]

[1,] 1 3

[2,] 2 4

19 Define Cbind() and Rbind() function

→ cbind(): This function is used to add additional column in a matrix.

rbind(): This function is used to add additional rows in a matrix.

20 define S3, S4 and reference classes in R.

→ * S3 class: S3 class is most popular in R. programming language, most of the classes are come predefined in R.

* Su class: Su class is an improvement over the S3 class. In R, we use the setclass() function to define a class.

* It is similar like S3 & S4 class and here we use setRefClass() function to define a class

21 Define factor with an example

→ Factor is a data structure which are used to categorize the data and to store it on multiple levels.

Ex: music ← factor (c("pop", "classic", "rock"))
print (music)

o/p: "pop", "classic", "rock"

22 List control structures in R programming

→ 01. if statement
02. if... else statement
03 if... else... if... else statement
04. nested if... else statement.

23 Mention the looping statements in R programming.

→ 01. while loop
02. For loop
03. repeate loop

24 What is if-else in R?

→ In if-else, condition is evaluated, if condition is true then if block or true block is executed. otherwise false block or else block is executed.

25 Mention Boolean operators in R with example.

→

Operators	Example	
&	4==3 & 1==1	False
|	1==1 | 4==3	TRUE
!	4==3 ! 4==4	False

26 Define recursion

→ A function that calls itself is called recursion function and this technique is called recursion.

27 Define lapply() and sapply() function

→ * lapply(): This function is used to apply a function over a list or matrix / dataframe / factor / array but this function is mainly made for list

* Sapply(): Sapply function is used to apply functions to each elements of list / matrix / dataframe / factor / array.

28 Define mapply() and tapply() function.

→ * mapply(): This function can be applied over a several list simultaneously.

* tapply(): This function is used to apply a function over a subset of vectors given by a combination of vector.

29 Define apply() function with example

→ This function applies a given agrigate function over a mangin of a given data structure.

Ex: a ← matrix (c(1,2,3,4), nrow=2, ncol=2)

print(x)

apply(x, 1, sum)

o/p: [1] 1 3

[2] 2 4

4 6

30 Define user defined function

→ It is a custom function created by the user to perform a specific task or operation

31 Mention Arithematic and logical operators in R

→ Arithematic operators

Addition	+
Subtraction	-
Multiplication	*
Division	/
Modulo	%/%
Exponential	^

Logical operators

Logical And	&
Logical or	|
Logical not	!

32 What is return statement?

→ A return statement in R is used to specify the output value of a function. it indicates the end of the functions execution and returns the result to the caller

database using dbplayr package.
* Using the flex dashboard package in R, you can create interactive web apps with a few line of code. you can also deploy apps on servers and develop video games.
* R is seventh rank of all programming language it is first rank language in data analysis.

01 Explain the features and applications of R programming
→ * It is open source programming language, you can install R for free.
* Integrated with other languages:
R has various data structures and operators, it can be integrated with other programming language like C, C++, java. and python, HDFS (Hadoop File system) FORTRON, TS
* Extensive packages and libraries:
R consists of various in built packages this makes reporting the results of an analysis easier by using R. There are more than 15000 packages in R.
* Comprehensive Statical analysis:
R is programming language that supports both procedural as well as object oriented programming.
* Platform independent:
R programming is platform independent i.e it can run on any operating system and in any software environment without any hassle.

* Open source and free
R is an open-source programming language with various statistical and graphical techniques including linear and non-linear modelling, time series analysis, clustering etc
* powerful data handling and storage:
R efficiently handles and stores data, supporting various data types and structures, including vectors, matrices, data frames and lists.
* Robust community and support:
R has a vibrant and active community that provides extensive support through forums, mailing lists, and online resources.
* Advanced data visualization
R integrates with various data source. R programming most widely used in science data.

Applications of R programming
* Data science and analytics:
R is used to analyze data, create visualizations, and generate reports
* Academics research
Research use R to create reproducible research and perform statistical analysis
* Bio informatics:
R is used to analyze genomic data. The bio conductor project within R provides packages and function for next generation

02 Describe operators in R programming with suitable example.

01. Arithmetic operators

→

Operator	Description
+	Addition of two numbers
-	Subtraction of two numbers
*	Multiply two numbers
/	Divides two numbers
%.%	To find the remainder
^	finding exponent of a number

Ex:

```
a ← 4
b ← 2
cat ("addition of numbers :", a+b, "/n")
cat ("Subtraction of numbers :", a-b, "/n")
cat ("multiplication of numbers :", a*b, "/n")
cat ("Division of two numbers :", a/b, "/n")
cat ("modulus is :", a %.% b, "/n")
cat (":power operator :", a^b)
```

O/P

```
addition of numbers : 6
Subtraction of numbers : 2
multiplication of numbers : 8
Division of two numbers : 2
modulus is : 0
power operator : 16
```

02. Assignment operator:

assignment operators in R used to assigning values to various data objects in R

In R there are 3 kinds of assignment operators:

01. Using equal operator (=):

By using equal operator you can assign right side value to the left side variable.

Syntax: variable_name = value

Ex: v1 = c(1,2,3)

02. Using left assignment / leftward operator (← or <<—)

By using left assignment operators you can assign right side value to lefthand side variable.

Syntax: variable_name ← value

Ex: v1 ← c(1,2,3)

03. Using Right assignment / rightward operator (→ or →>)

By using right assignment operator, you can assign left side value to the righthand side variable.

Syntax: value → variable_name

ex: c(1,2,3) → v1

03. Relational operators:

These operators are used to compare between values each element of the first vector is compared with corresponding element of the second vector.

The result of the comparison is boolean value (TRUE / FALSE)

operators are: >, <, ==, >=, <=, !=

04. Logical operator:

It is applicable only to vectors of type logical numeric and complex.

zero is considered as FALSE non-zero numbers are taken as TRUE

Each element of first vector is compared with corresponding element of second vector.

Operators are : $, !, |

05. miscellaneous operators:
These operators are used to specific purpose.

operators	Description
:	colon operator, creates series of numbers in sequence.
%in%	used to identify if an element belongs to a vector

03 Define vector. Explain different methods of defining vectors in R.

→ A vector is a simply a list of a items that are of same type, to combine the list of items to a vector, use c() and seperate the elements by comma.

methods 1 : c() function:
c() is the most common method to create a vector

```
x ← c(1,2,3,4,5)
```

02: vector() function.
vector() creates a vector with specified mode and length

```
x ← vector("numeric", 5)
x ← c(1,2,3,4,5)
```

03: colon operator (:)
The colon operator creates a sequence of numbers

```
x ← 1:5  # equivalant to c(1,2,3,4,5)
```

04. Sequence() function:
Sequence() generates a sequence of numbers

```
x ← Sequence(5)
```

o/p : 1 2 3 4 5

05. Rep() function:
rep() repeats a value or vector
a ← rep(1,5)
o/p 1 1 1 1 1
06. Seq() function:
Seq() generates a sequence with specified start, end and increment
a ← seq(1,5, by=1) o/p: 1 2 3 4 5
07. Character vector:
Create a character vector using c() with quotes
a ← c("apple", "Banana", "Cherry")
08. logical vector:
Create a logical vector using c() with TRUE / FALSE
a ← c(TRUE, FALSE, TRUE)
09. Factor vector:
Create a factor vector using factor()
a ← factor(c("male", "female", "male"))

04 Explain usage of matrices in R programming with suitable examples.

→ *Using matrices we can access more than one row and one column.

ex: m1 ← matrix(c(1,2,3,4,5,6,7,8,9), nrow=3, ncol=3)

print(m1)

	[1]	[2]	[3]
[1]	1	4	7
[2]	2	5	8
[3]	3	6	9

print(m, [c(1,2),])
o/p: 1 4 7
2 5 8

print(m, [c(1,2)])
o/p: 1 4
2 5
3 6

* Using cbind() and rbind() to add additional rows and columns.

```
m2 ← cbind(m1, c(10, 11, 12))
print(m2)
```

O/p

1	4	7	10
2	5	8	11
3	6	9	12

```
m2 ← rbind(m, c(10, 11, 12))
print(m2)
```

O/p

1	4	7
2	5	8
3	6	9
10	11	12

* Using matrix we can perform many operations like:

- matrix transpose:

Transpose of a matrix in R is an operation in which we convert the rows of matrix into columns and columns of the matrix into rows.

Ex: $m = \begin{bmatrix} 2 & 5 & 8 \\ 6 & 7 & 9 \end{bmatrix}$ $mT = \begin{bmatrix} 2 & 6 \\ 5 & 7 \\ 8 & 9 \end{bmatrix}$

- Identity matrix:

It is where diagonal consists of 1's and other elements are all zeros. it is also called as unit matrix and elementary matrix.

$I_2 = I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}_{2X2}$

- matrix addition and subtraction:

In R, '+' operator used to add two matrices

```
A = matrix(c(1:6), nrow=2, ncol=3)
B = matrix(c(7:12), nrow=2, ncol=3)
print(A+B)
```

O/p:

8	12	16
10	14	18

In R, '-' operator used to subtract two matrices

```
A = matrix (c (1:6), nrow=2, ncol=3)
B = matrix (c (7:12), nrow=2, ncol=3)
print (A-B)
```

o/p -6 -6 -6
     -6 -6 -6

- matrix multiplication

In R, by using '*' operator is used to multiply the two matrices.

```
A = matrix (c (1:4), nrow=2, ncol=2)
print (A)
B = matrix (c (1:4), nrow=2, ncol=2)
print (B)
print (A*B)
```

o/p $A = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$ $B = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$ $A*B = \begin{bmatrix} 1 & 9 \\ 4 & 16 \end{bmatrix}$

05 Write a R pgm to demonstrate graphic functions plot(), hist(), linechart(), pie(), barplot()

→ i> plot()

```
x ← c(1, 2, 3, 4, 5)
y ← c(4, 6, 8, 2, 10)
plot (x, y,
      main = "plot example",
      xlab = "x-axis label",
      ylab = "y-axis label",
      col = "red")
```

ii> hist()

```
x ← c(21, 23, 56, 90, 20, 7, 94, 12, 57, 84, 76, 69, 45, 92,
      34, 32, 49, 55, 57, 98)
```

```
hist (x,
      main : " Histogram example",
      xlab : " values",
      border = " blue",
      col : " orange",
      col. lab : " red",
      col. main : " green")
```

iii> linechart()

```
x ← c(1,2,3,4,5)
y ← c(4,6,8,2,10)
plot (x, y, type="1",
main : " linechart",
xlab = "x-axis label",
ylab = "y-axis label",
col = "Red")
```

iv> pie()

```
x ← c(150, 450, 250, 100, 50, 90)
names(x) ← c(" DS", " JAVA", " PYTHON", "C#", "C++", "C")
pie (x,
      labels = names(x),
      col = " orange",
      main = " programming",
      radius = -1
      col. main = " green")
```

v> boxplot()

```
x ← c(5,7,10,2,6,8,8,1,2,5,7,3,3,4)
boxplot(x,
        main: "Boxplot",
        xlab: "x",
        col: "green",
        border = "pink")
```

vi> Scatter plot()

```
car.age ← c(5,7,8,2,9,11,6)
car.speed ← c(99,86,88,111,87,78,86)
plot(car.age, car-speed,
     main: "Scatterplot",
     xlab = "Car-age"),
     ylab = "Car-Speed"),
     xlim = c(1,10),
     ylim = c(80,110),
     pch = 15,
     col = "blue")
```

06 describe any five basic plotting functions in R with ex

→ * In data analysis data visualization is important role, data visualization is a technique used to delivery the data in visual such as graphs, charts and maps etc

* Data visualization is usefull as, it helps easy understanding of large quantities of data and to make better decision it

various plotting functions in R are.

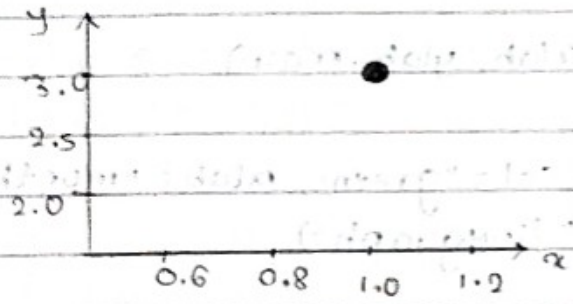
01. plot():

The plot() function is used to draw points in a diagram. This function takes parameter for specifing points in the diagram.

→ parameter 1. specifies points on the x-axis

→ parameter 2. specifies points on the y-axis

ex:- plot(1,3) syntax:- plot(v1, v2).



02. Barplot:

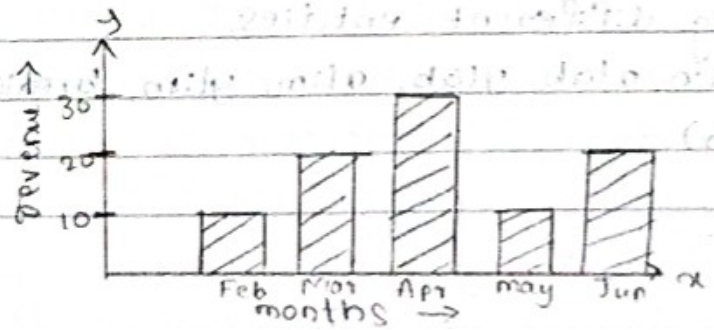
Barplot is one of the most efficient way of representing the data. It can be used to summarize large data in visual from. The barchart is used to summarize a set of categorical data.

*In R, to create a bar chart R provide barplot() function

syntax: barplot(H, names.arg, xlab, ylab, main, col = " ")

ex:-

```
H ← c(10, 15, 30, 5, 20)
M ← c("Feb", "Mar", "Apr", "May", "Jun")
bar1 ← barplot(H, names.arg = M, xlab = "months", ylab = "Revenue"
, col = "blue", main = "Revenue barchart", border = "red")
print(bar1);
```



03. Linegraph:

A line graph is a pictorial representation of information which is changes continously over time.

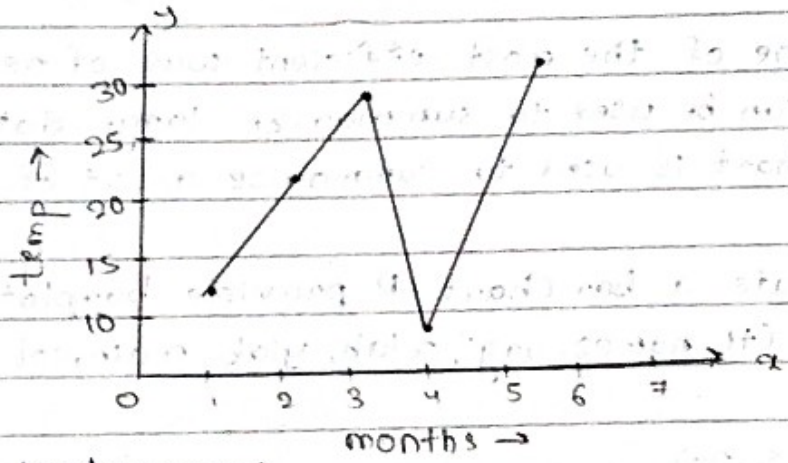
* In line graph, the points connecting the data to show the continoas change.
* A line in line graph can move up or down based on the data
* for linegraph, R provides plot() function and add parameter as type = 'l'

Syntax: plot (v, type, col, xlab, ylab, main)

ex: v ← c(13, 22, 28, 7, 31)

line1 ← plot (v, type = "l", col = "green", xlab = "month", ylab = "temp", main = "linegraph")

print (line1)



04. Histogram:

A histogram is a type of bar chart which shows the frequences of the number of values which are compared with set of values ranges.

* Histogram is used for distrubution, where as bar chart is used for comparing different entities.

Syntax: hist (v, main, xlab, ylab, xlim, ylim, breaks, col, border)

05. Pie Chart:

pie chart is a representation of values in the form of slices with different colors.

* Slices are labeled with description and the number of corresponding to each slice.

Syntax: pie(x, labels, radius, main, col, clockwise)

Ex: x ← c(20, 65, 15, 50)

labels ← c("India", "America", "Srilanka", "nepal")

pie1 ← pie(x, labels, main = "pie chart", radius = 1, col = "blue", clockwise = "TRUE")

07 Write a note on special values in R.

→ R language supports special several values and these special values can be used to mark abnormal or missing values in a vector, array or other data structure.

01. Infinity:

When a number is too large for R to represent. value is determined to be infinite. This value is represented by the special object "Inf" which is case sensitive because it represents a numeric value. Inf can be assossiated with only numeric vectors.

* is.infinite() function is used to check if the element of vector containg containing infinite elements are present or not.

Syntax: is.infinite(x)

ex: n ← c(10, Inf, 20, Inf)

o/p : False True False True.

* is.finite() in R, this function used to check if the elements of a vector are finite values or not.

Syntax: is.finite(x)

02. NaN : (Not a Number)

A Nan value in R represents Not a Number, it is basically any numeric calculation with an undefined result, such as %. this exists only in vector with numeric data type.

ex: a = c(2.5, 8, 20, Nan, 31, NaN)

o/p : 2 5 8 20 Nan 31 Nan

03. NA (Not Available)

In R, missing values are represented by the symbol NA

ex: a = c (2, 4, 8, NA, 10, NA)

o/p : 2 4 8 NA, 10 NA

04. NULL:

In R null represent NULL object. NULL is mainly used to represent the list with zero length. NULL is not same as NA, NaN & Inf.

ex: a = c()

print (a) o/p : NULL

08 Explain types of vector

→ 01. Numeric vector : These vectors contains the numeric values.

ex: v1 ← c (4, 5.5, 2, 3)

typeof(v1) print (v1)

numeric 4, 5.5, 2, 3

02. Integer vector : These vectors contains integer values

ex: v2 ← c (4L, 5L, 2L, 3L)

type of (v2) print (v2)

integer 4 5 2 3

03. Character vector : These vectors contains alphabets, numbers and special characters.

ex: $v_1 \leftarrow c("BCA", "MCA", 57)$

typeof(v1) print(v1)

Character "BCA" "MCA" 57

04. Logical vector: logical vectors in R contains boolean values such as TRUE, FALSE and NA (null)

ex: $v_1 \leftarrow c(TRUE, FALSE, FALSE, NA)$

typeof(v1) print(v1)

09 Explain the various matrix operations in R.

→ 01. Matrix transpose: Transpose of a matrix is an operation in which we convert the rows of matrix into columns and columns of the matrix into rows.

Transpose of a matrix in R can be performed in 2 ways.

* t() is used to find the transpose.

*By iterating over each value using loops.

ex: $m = \begin{bmatrix} 2 & 6 \\ 0 & 7 \end{bmatrix}$ $mT = \begin{bmatrix} 2 & 0 \\ 6 & 7 \end{bmatrix}$

02. Identity matrix:

It is a matrix which is square matrix where diagonal consists of 1's and other elements are all zeros it is also called as unit matrix and elementary matrix.

ex: $I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}_{2\times2}$

* diag() function is used to create identity matrix.

Syntax: diag(num)

03. Scalar multiplication:

In scalar multiplication every element of a matrix is multiplied by a given scalar.

Ex: 2*m1 $m_1 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ o/p: $\begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}$

04. Matrix addition and subtraction:
matrices of the same order. the their sum A+B is a matrix and each element of that matrix is the sum of the corresponding elements i.e A+B = [aij + bij]

ex: $A\begin{bmatrix}2 & 5\\ 6 & 5\end{bmatrix}$ $B=\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix}$ $A+B=\begin{bmatrix}3 & 7\\ 9 & 9\end{bmatrix}$

05. Inverse matrix:
The inverse of matrix is found using following formula
$A^{-1} = adj(A) / det(A)$
ex: Solve (A)

$A=\begin{bmatrix}5 & 6\\ 5 & 4\end{bmatrix}$ $A^{-1}=\begin{bmatrix}4 & -6\\ -5 & 5\end{bmatrix}$

10 Explain R-arrays with examples
→ In R, arrays are the data structure which allows us to store same type of data in more than two dimension
Syntax: array_name ← array (vector, dim = c(nrow, ncol, nmatrix))
ex: a ← array (c(1:12), dim = c(2,3,2))

$\begin{bmatrix}1 & 3 & 5\\ 2 & 4 & 6\end{bmatrix}$ $\begin{bmatrix}7 & 9 & 11\\ 8 & 10 & 12\end{bmatrix}$

first matrix second matrix

*access array elements:
a ← array (c(1:12), dim = c(2,3,2))
a[1,3,2] o/p: 5

* Check if element exist:
11 %in% a TRUE
20 %in% a FALSE

* Length of array.
Syntax: length (arrayname)
length (a) o/p: 12

* Amount of rows and columns:

dim(arrayname) # syntax

dim(a) o/p : 2 3 2

* loop through an array

you can loop though the array items by using for loop

```
for(x in a)
{
  print(x)
}
```

o/p 1 2 3 4 5 6 7 8 9 10 11 12

11 Explain R lists with examples:

→ In R, lists are one-dimentional heterogenous data structure. The list of vectors, list of matrices list of characters, list of functions ...

A list is a collection of data which is ordered and changable.

Syntax: listname <- list(elements of list)

l1 <- list(10, 20, 30, 40)

* Access list elements

Syntax: listname[element inder]

l1[2] o/p : 20

* modify list element:

l1[3] ← 26 print(l1) 10 20 26 40

* remove item from list:

listname(-inder)

l1[-3] o/p : 10 20 40

* length of list:

Syntax : length(listname)

ex : length(l1) o/p: 3

* Check element exist in list
ex: 30 %in% d1 TRUE
100 %in% d1 FALSE
* range of indexes
d1[1:3] 10 20 30
* join two list
d2 ← list(50,60,70,80)
c(d1, d2)
O/p: 10 20 40 50 60 70 80

12 Explain R dataframe indetail
→ * A data frame is a two dimentional data structure which can store data in Tabular formal.
* Data frames have rows and columns each column can be different datatype.
* In R, we use data.frame() function to create data frames.
Syntax: dataframe-name ← data.frame(first-name=c(v1,v2), second-column-name=c(v1,v2),...)
ex: df ← data.frame(name=c("pranam","param","Bhoomi") age(20,21,22))

O/p	Name	Age
1	pranam	20
2	param	21
3	Bhoomi	22

* access data frame column.
df[1] "pranam" "param" "Bhoomi"
* length of data frame in R
In R, use the length() function to find the number of columns in or dataframe.
length(df) O/p: 2

* Combine dataframe:

* In R, we use the rbind() function and cbind() function to combine two data frames together.

* Using rbind() function we combine two dataframes vertically the column name of the two dataframes must be same.

* Cbind() function combines two or more data frames horizontally

14 Explain control statements in R.

→ These are the expressions used to control execution and flow of a program based on the conditions provided in the statement.

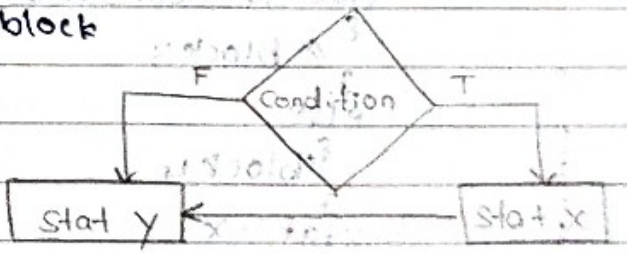
Types:

* if statement
* if... else statement
* if... else... if... else statement
* nested if... else statement.

01. if statement

Condition to be evaluated if condition is true if block is executed otherwise it will skip the if block

Syntax: if (expression)
{
// if block or true block
}
stat-y

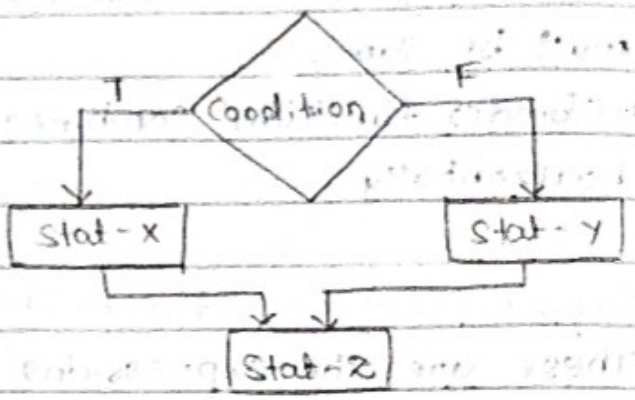


eg: a ← 3
if (a > 0)
{ print ("a is +ve")
}

02. if... else statement:

Condition is evaluated, if condition is true then if block is executed otherwise false block is executed.

Syntax: if (expression)
{ // true block
stat-x
}
else
{ // stat-y
}
stat z

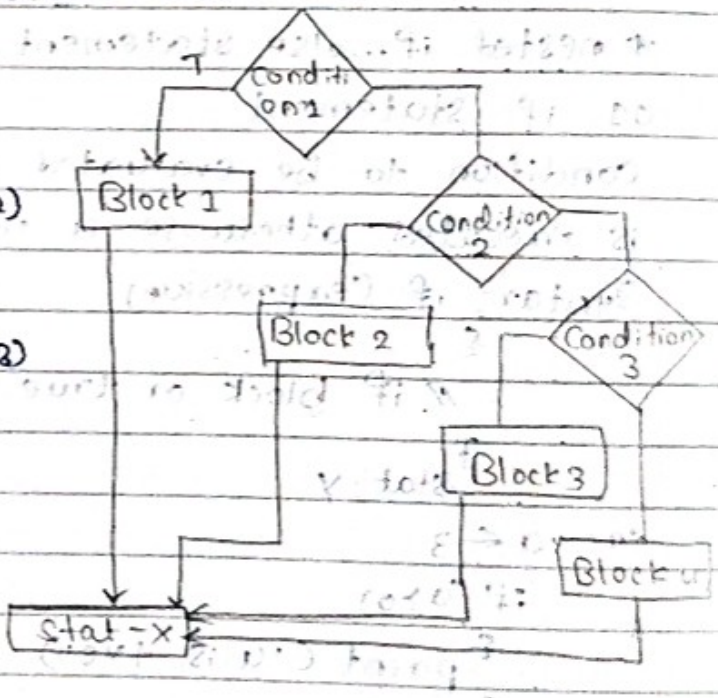


ex: a ← 6
if (a > 0)
print ("a is +ve")
else
print ("a is -ve")

03. if... else... if... else statement

This statement used to ~~ver~~ test various condition in a single if..else..if statements.

Syntax: if (Condition 1)
{ // block 1
}
else if (Condition 2)
{ // block 2
}
else if (Condition 3)
{ // block 3
}
else
{ // block 4
}
stat -x

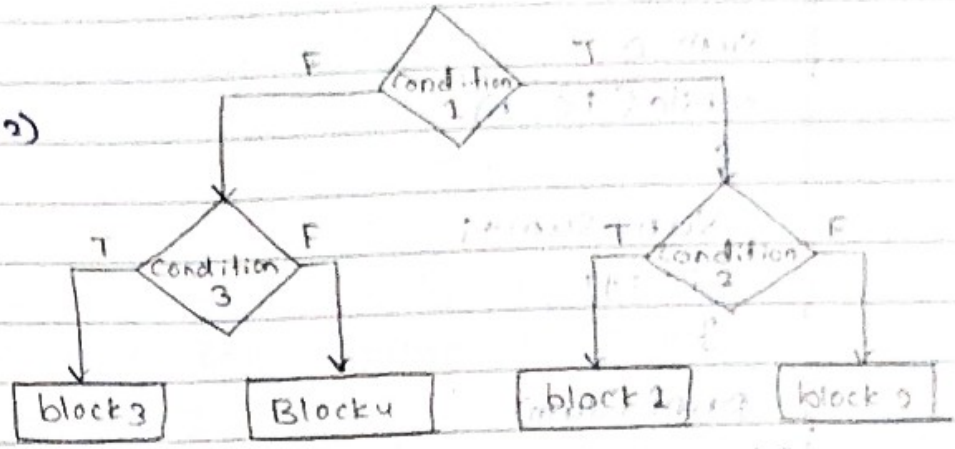


04. Nested if...else statement

you can have nested if else statement inside if else block in R. this is called nested if else statement

Syntax:

```
if (condition 1)
  { if (condition 2)
     {
       block 1
     }
    else
     {
       block 2
     }
  }
else
  { if (condition 3)
     {
       block 3
     }
    else
     {
       block 4
     }
  }
```



Explain the loops in R

→ loops are use to repeate the execution of block of code, loop helps you to save the time and avoid repe-atable block of code and to write cleaner code

⇒ advantages: * code reusability
* simplicity
* improve overall efficiency

01. while loop.

while loop is used when you don't know the exact number of time the block of code to be repeated.

Syntax: while (test - condition)
{ // block of code
}

ex:

```
num: readline(prompt:" enter a n value:")
n:as integer(num)
i:1
sum:0
while(i<=n)
 {
  sum:sum+i
  i:i+1
 }
print(sum)
```

o/p: Enter a n value: 5
15

02. For loop

A for loop is used when you know the exact number of times a block of code to be repeated.

Syntax:

```
for (value in sequence)
    {
      // block of code
    }
```

03. repeat loop:

It is used to execute code block multiple times the repeate loop doesnot have any condition to terminate the loop, you need to put exit * condition explicitly with a break statement inside the loop

Syntax:

```
repeat {
    if (stop-condition)
      {
        break
      }
  }
```

```
eg: x == 1                    o/p: 1
    repeat {                       2
      print(x)                     3
      if (x>4)                     4
       {                           5
        break
       }
      x=x+1
    }
```

16 Write a R pgm to find the fiborracci of a number using recursion function

```
fib <- function(n)
 {
   if(n<=1)
    {
     return(n)
    }
   else{
     return(fib(n-1)+fib(n-2))
    }
 }
total_terms <- as.integer(readline(prompt:"How many terms ?"))
if (total_terms <=0)
{
 print("please enter a +ve integer")
}
 else{
   print("Fibonacci sequience is:")
     for(i in 0(total_terms-1))
      {
        print(fib(i))
      }
 }
```

O/p: How many times ? 5
Fibonacci series is:
0 1 1 2 3

17 Explain R language operators

→ * Arithematic operators:

These operators are used to carryout, mathematical operations like addition, multiplication, division, subtraction

Operators	Description
+	Adds two vectors
−	Subtract second vector from first
*	multiplies both vector
/	divide the first vector with second
%:%	Give the remainder of first vector into 2nd.
^	first vector is raised to component of second vector.

* Relational vector: operator:

These operators are used to compare between values. each element of ve first vector is compared with corresponding element: and result we get boolean value operators are : <, >, >=, <=, ==

* Logical operators:

It is applicable only for vectors of logical, numeric and complex. zero is considered as FALSE, non zero numbers are taken as true.

logical operators are: &, |, !

* Assignment operators: are used to assigne assign values

operators	discription
<−, <<−, =	left assignment
→, →>	Right assigorment.

+ miscellaneous operators:

operators	Description
:	Colon operators, creates series of numbers in sequence.
%in%	used to identify if an element belog to a vector or not.

18 Define user-defined functions, write the syntax, and an example of defining user defined functions in R.

→ It is a custom function created by the user to perform a specific task or operation.

Syntax:

```
function-name <- function (arg1, arg2...)
{
 function-body
}
{
 function-name (val1, val2...)
}
```

or:

```
f1 <- function()
 {
 for(i in 1:10)
 {
 print(i^2)
 }
 }
f1
```

O/P:

```
1
4
9
16
25
36
49
64
81
100
```

19 What is if...else in R? write an R pgm illastrate with if..else statement and how does it operate on vectors of variable length.

→ # Check if x is positive or negative number

```
if (x>0) {
    print (" x is positive number")
} else {
    print (" x is a negative number")
}
```

if else returns a value with the same shape as test which is filled with elements selected from either yes or no depending on whether the element of test is TRUE or FALSE

11 differentiate betn paste() and cat() function.

→ paste()

* return a character vector containing the connected strings

cat()

* print the output directly to the console, without returning a value

* allows you to specify a seperator between the concatenated strings: allows for separator specification	* does not allow for seperator specification
* Concatenate strings and returns the result	* Concatenates strings and prints the result directly to the console.
* Syntax: paste("any string", variable names))	* Syntax: cat("Any string", variable names).

12 differentiate between print and paste function

→

Print()	paste()
* prints output directly to the console	* Concatenates strings and returns the result
* primarily used for printing output to the console	* primarily used for concatenating the strings
* return the value printed. but is often used for its side effect	* Returning the concatenated string
* Typically used for displaying results or messages	* often used for creating strings, file name, or path

13 Define Binary Search Tree (BST)

→ It is collection of nodes interconnected with branches a binary tree in which:

* Each node has a unique value
* The left subtree of a node contains only values less than the node value and right subtree contains only values greater than the nodes value.

07 Create a note on error handling in R

→ Error handling is a process in which we deal with unwanted error or anomalous error which may cause abnormal termination of the program during its execution.

*Stop():

Stop() function is used to throw an error, it stops execution of the current statement prints the message given as it's argument and then returns to the top level prompt.

Syntax: Stop("String msg")

ex:
```
for (i in 1:10)
  {
   if (i != 5)
    { cat ("Iteration number", i)
    }
   if (i == 5)
    { stop ("I was equal to 5")
    }
  }
```

o/p: Iteration number 1

Iteration number 2

Iteration number 3

Iteration number 4

I was equal to 5

* Warning ():

This function generates a warning message before someone takes an action / loss of data it is like attention to the user

Syntax: warning ("string_msg")

ex: warning message example

```
for(i in 1:10)
  {
    print (paste("i=", i)
    if (i==5)
       waring ("i is at 5")
  }
```

o/p: i=1 warning message:

i=2 i is at 5

i=3 i=6

i=4 i=7

i=5 i=8

i=9

i=10

* message (): It generates diagnostic message

Syntax: message (" string_msg")

ex: for (i in 1:5)

```
  {
    print (paste(" i=", i))
    if (i==3)
      message ("i is at 3")
  }
```

o/p: i=1

i=2

i=3

i=3 at 3

i=4

i=5

* try(): It is used to handle errors and expressions that may occurs during the execution of code it allows you to execute a block of code and catch any errors that might arise, preventing the code from stoping and allows you to handle the errors efficiently.

Syntax: try(expression, silent = FALSE)

* tryCatch():

you can use a tryCatch() function in R to return a value of some expressions or produce a custom message if a warning or error is encountered.

Syntax: tryCatch({
    // expression
}, warning = function(w){
    // code that handles the warning
}, error = function(e){
    // code that handles the error
},
finally = function(f){
    // finally code
})

* withCallingHandlers():

It is an alternative to tryCatch(), the withCallingHandlers() is varient of tryCatch(), the only difference is tryCatch() deals with existing handlers (built-in errorep exception functions) while with calling handlers() deals with local handlers (user defined exception functions).

}

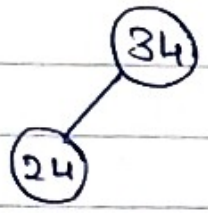
o/p: 0.5

09 Define binary search tree. draw BST for given values

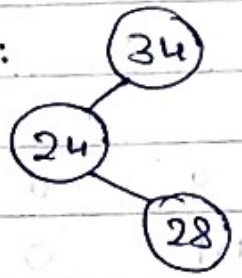
34 24 28 65 9 56 17 23

→ It is collection of nodes interconnected with branch -es in this tree, the left child are less than the node value and the values in the right child are greater than the node value.

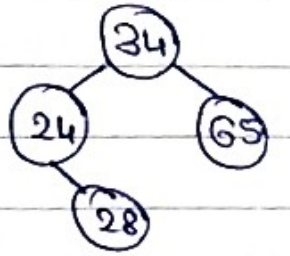
Step 1:



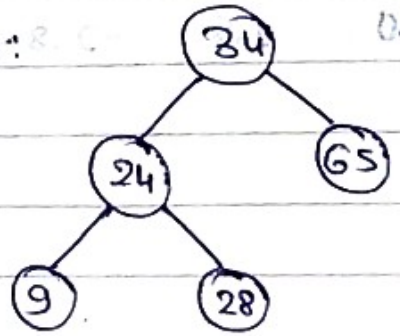
Step 2:



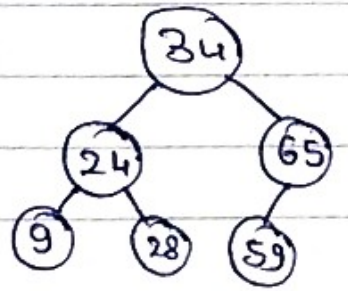
Step 3:



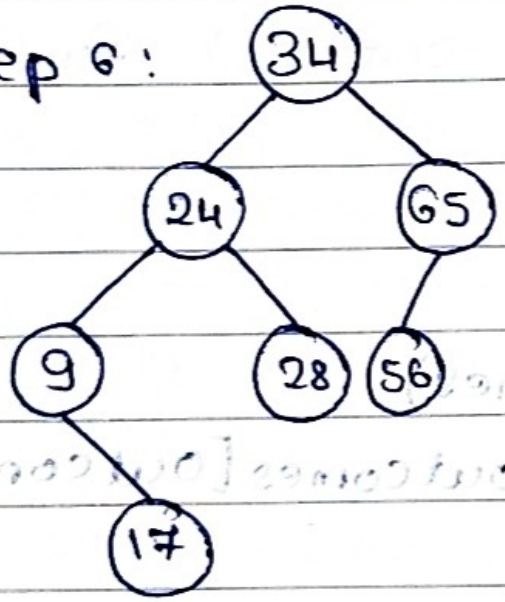
Step 4:



Step 5:



Step 6 :



Step 7 :

